

Retail RAG Assistant (Thời trang, song ngữ VI/EN)

RAG-based chatbot hỗ trợ CSKH cho shop thời trang: trả lời câu hỏi về sản phẩm, chính sách đổi trả, vận chuyển, khuyến mãi — dựa trên dữ liệu từ [Fashion Product Images \(Small\) - Kaggle](#).

Kiến trúc

```
User Query -> Embedding (BAAI/bge-m3) -> ChromaDB vector search
            -> Context (product + policy chunks)
            -> Groq LLM (Llama-3) -> Answer + Sources
```

Cấu trúc thư mục

```
data/
  raw/           # styles.csv gốc
  processed/     # products.csv (đã lọc + song ngữ) + policies/*.md
  eval/         # eval_set.csv, results.csv
src/
  ingest/       # xử lý data, build vector index (chạy 1 lần)
  rag/          # retriever, prompt, generator (runtime)
  eval/        # script đánh giá
api/           # FastAPI backend
frontend/      # Streamlit UI
```

Setup

```
python -m venv venv
source venv/bin/activate      # Windows: venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env         # điền GROQ_API_KEY
```

Pipeline (thứ tự chạy)

1. `python src/ingest/clean_products.py` — tạo `data/processed/products.csv`
2. Viết policy docs vào `data/processed/policies/*.md` (thủ công)
3. `python src/ingest/build_index.py` — build vector index vào `chroma_db/`
4. `uvicorn api.main:app --reload` — chạy backend
5. `streamlit run frontend/app.py` — chạy frontend
6. `python src/eval/run_eval.py` — chạy eval, xem kết quả

Đánh giá (metrics)

- **Retrieval:** Recall@1, Recall@3

- **Answer quality:** LLM-as-judge (relevance, correctness, faithfulness — thang 1-5)
- **Latency:** thời gian phản hồi trung bình end-to-end
- **Refusal accuracy:** % câu hỏi ngoài phạm vi (out-of-scope) được từ chối đúng cách

Docker

Chạy cả backend + frontend cùng lúc bằng Docker Compose:

```
cp .env.example .env # điền GROQ_API_KEY (bắt buộc trước khi build)
docker compose up --build
```

- Backend: <http://localhost:8000> (Swagger UI ở </docs>)
- Frontend: <http://localhost:8501>

Lần chạy đầu tiên, container backend sẽ tự chạy `build_index.py` nếu `chroma_db/` (volume `chroma_data`) đang trống — không cần chạy tay trước. Các lần `docker compose up` sau đó sẽ tái sử dụng index đã build (volume persist), khởi động nhanh hơn.

Cả 2 service (`backend`, `frontend`) dùng chung 1 image (build từ cùng `Dockerfile`) để đơn giản hoá — `frontend` override `entrypoint/command` để chạy Streamlit thay vì uvicorn.

Lưu ý (Windows/git): nếu gặp lỗi kiểu `entrypoint.sh: not found` hoặc `bad interpreter`, khả năng cao do git tự chuyển line ending của `entrypoint.sh` sang CRLF. Kiểm tra/sửa bằng:

```
git config core.autocrlf false
dos2unix entrypoint.sh # hoặc mở bằng VS Code, đổi CRLF -> LF ở góc dưới phải
```

Deploy lên HF Spaces / Render: cả 2 nền tảng đều build trực tiếp từ `Dockerfile` — với HF Spaces cần đổi `EXPOSE`/port sang `7860` (quy ước của Spaces) và thêm YAML header (`sdk: docker`) vào đầu `README.md`; với Render tạo Web Service trở tới repo, chọn "Docker" làm environment, khai báo `GROQ_API_KEY` trong phần Environment Variables. Vì mỗi nền tảng có quy ước riêng, nên xác nhận chọn nền tảng nào trước khi cấu hình chi tiết.

TODO / Trạng thái

- ☒ `clean_products.py` — lọc + song ngữ hoá data
- ☒ Policy docs (5 file: return, shipping, promotion, warranty, size guide)
- ☒ `eval_set.csv` (55 câu: product / policy / out-of-scope)
- ☒ `build_index.py` — embedding + ChromaDB
- ☒ `retriever.py`, `generator.py`, `prompt.py`
- ☒ FastAPI `/chat` endpoint hoàn chỉnh
- ☒ Streamlit frontend nối API
- ☒ `run_eval.py` — tính đủ 4 metrics
- ☒ Docker Compose (backend + frontend, auto-build index)
- ☐ Deploy thật lên HF Spaces / Render (cấu hình cụ thể theo nền tảng chọn)